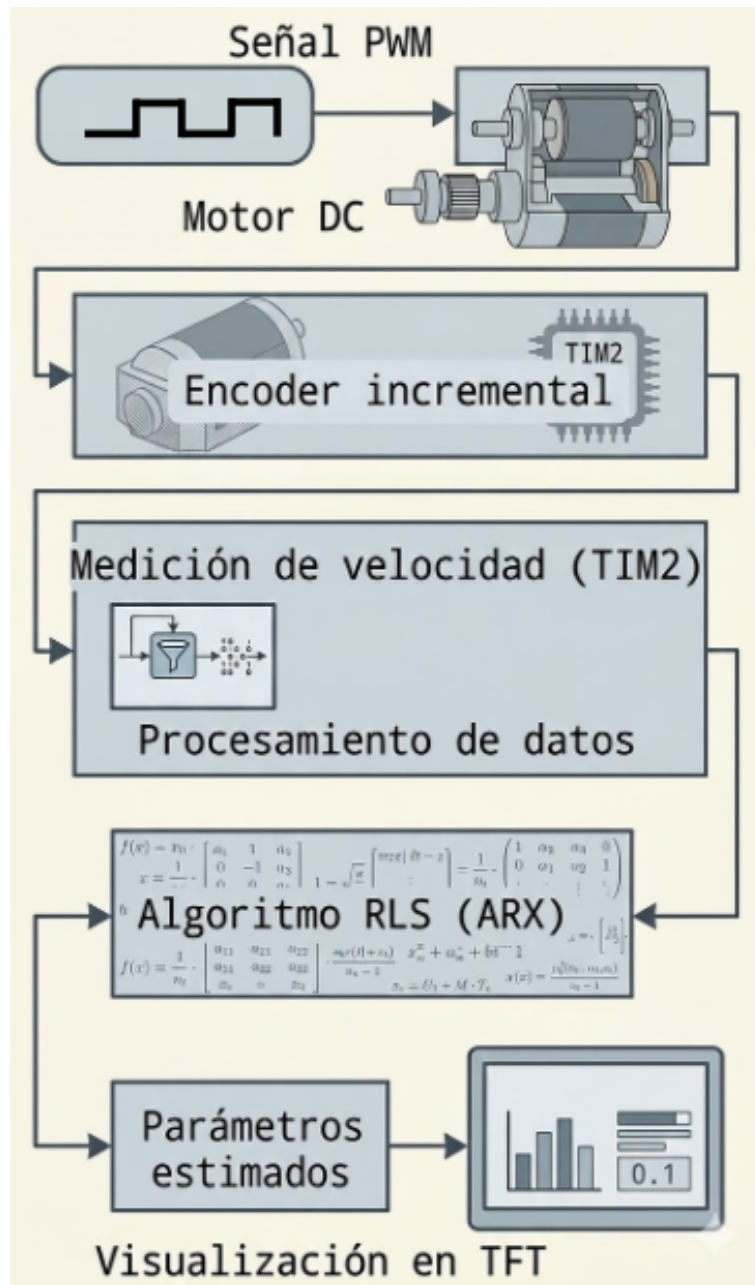


Módulo 5: quad_enc

Documentación didáctica de `quad_enc.c/h` y `quad_enc_if.c/h`
Proyecto STM32F405RGT6 - identificación de motor DC con RLS/ARX



Este módulo convierte al temporizador **TIM2** en una interfaz de lectura para un encoder incremental. Su función es entregar la **posición acumulada en ticks** y servir como base para estimar la **velocidad del motor**, dato que después alimenta al algoritmo RLS/ARX del proyecto.

Periférico	Pines	Modo	Uso en el proyecto
TIM2	PA0 = CH1 PA1 = CH2	Encoder mode 3	Conteo en cuadratura, medición de posición y cálculo de RPM cada 10 ms

Idea clave para el alumno: el microcontrolador no cuenta pulsos por software. Se aprovecha el hardware de TIM2 para decodificar la cuadratura y mantener el conteo automáticamente, lo que reduce carga de CPU y mejora la robustez de la medición.

1. Relación entre driver e interfaz

La organización del proyecto separa el acceso directo al hardware de la forma en que el módulo se usa desde la aplicación.

Archivo	Rol	Qué contiene
quad_enc.h	Header del driver	Prototipos públicos: inicialización, lectura de conteo, reset y cálculo de velocidad.
quad_enc.c	Driver hardware	Configuración de GPIOA y TIM2 en modo encoder mediante registros CMSIS.
quad_enc_if.h	Interfaz abstracta	Struct con punteros a función para desacoplar aplicación y driver.
quad_enc_if.c	Instancia concreta	Asocia la interfaz quad_STM32 con las funciones reales del driver.

- La aplicación invoca **quad_STM32.init()**, **quad_STM32.get_count()** y **quad_STM32.reset()** sin depender del nombre interno de las funciones del driver.
- Este patrón facilita enseñar **abstracción de hardware**: el código de alto nivel ve una interfaz, mientras que la implementación específica queda encapsulada.
- También permite sustituir el backend en el futuro: por ejemplo, otra tarjeta o incluso una fuente de datos simulada.

Fragmento representativo de la interfaz:

```
typedef struct
{
    void      (*init)(void);
    int32_t   (*get_count)(void);
    void      (*reset)(void);
    int32_t   (*get_speed_tps)(float Ts);
} quad_enc_if_t;

extern const quad_enc_if_t quad_STM32;
```

2. API pública del módulo

Función	Propósito	Comentario didáctico
encoder_tim2_init()	Configura PA0/PA1 y TIM2 en modo encoder.	Debe ejecutarse una sola vez durante el arranque.
encoder_get_count()	Lee el valor actual de TIM2->CNT.	Representa la posición acumulada en ticks.
encoder_reset()	Escribe cero en TIM2->CNT.	Permite convertir posición acumulada en posición por ventana de muestreo.
encoder_get_speed_tps(Ts)	Calcula ticks por segundo mediante diferencia discreta.	Útil como utilidad general, aunque en este proyecto principal no es la ruta usada para las RPM.

3. Configuración de TIM2 en modo encoder

La función `encoder_tim2_init()` sigue una secuencia clásica de inicialización hardware: habilitar relojes, configurar pines, ajustar registros del temporizador y finalmente arrancar el contador.

Pas o	Registro / acción	Propósito técnico
1	RCC->AHB1ENR = GPIOAEN RCC->APB1ENR = TIM2EN	Activa los relojes del puerto A y del temporizador 2.
2	GPIOA->MODER = AF para PA0 y PA1	Conecta físicamente los pines al periférico TIM2, no como GPIO común.
3	GPIOA->AFR[0] = AF1	Selecciona la función alternativa 1, correspondiente a TIM2_CH1/TIM2_CH2.
4	GPIOA->PUPDR = pull-up	Ayuda si la salida del encoder es colector abierto o si el cableado es susceptible a ruido.
5	TIM2->ARR = 0xFFFFFFFF TIM2->CNT = 0	Define el máximo rango del contador y arranca en cero.
6	TIM2->CCMR1: CC1S=01, CC2S=01	Mapea IC1 e IC2 a TI1 y TI2 para que ambas entradas alimenten el decodificador de cuadratura.
7	TIM2->CCMR1: IC1F=4, IC2F=4	Introduce un filtro digital moderado para reducir conteos falsos por ruido.
8	TIM2->CCER: CC1E=1, CC2E=1	Habilita ambos canales de captura.
9	TIM2->SMCR: SMS=011	Selecciona encoder mode 3 : se cuenta usando los dos canales en cuadratura completa.
10	TIM2->CR1: CEN=1	Inicia el temporizador.

Nota importante: en encoder mode 3 el hardware interpreta el desfase entre A y B para decidir el sentido de giro. Por eso el contador puede aumentar o disminuir sin necesidad de una ISR por flanco.

Fragmento central del driver:

```
TIM2->CCMR1 &= ~(TIM_CCMR1_CC1S | TIM_CCMR1_CC2S);  
TIM2->CCMR1 |= (1u << TIM_CCMR1_CC1S_Pos) | (1u << TIM_CCMR1_CC2S_Pos);  
  
TIM2->CCMR1 &= ~(TIM_CCMR1_IC1F | TIM_CCMR1_IC2F);  
TIM2->CCMR1 |= (4u << TIM_CCMR1_IC1F_Pos) | (4u << TIM_CCMR1_IC2F_Pos);  
  
TIM2->SMCR &= ~TIM_SMCR_SMS;  
TIM2->SMCR |= (3u << TIM_SMCR_SMS_Pos); // encoder mode 3
```

4. Cómo se obtiene la velocidad

El módulo ofrece dos caminos conceptuales para estimar velocidad:

- **Camino A - genérico:** usar `encoder_get_speed_tps(Ts)`, que calcula la diferencia entre la cuenta actual y la anterior, luego divide entre el periodo de muestreo T_s y devuelve **ticks por segundo**.
- **Camino B - usado en el proyecto:** leer los ticks acumulados cada 10 ms dentro de `TIM4_IRQHandler`, convertirlos a RPM con los datos mecánicos del sistema y después resetear el contador.

4.1 Función de utilidad `encoder_get_speed_tps(Ts)`

La implementación mantiene internamente una variable estática **last**. En cada llamada calcula:

$\text{delta} = \text{now} - \text{last}$ y $\text{speed} = \text{delta} / T_s$

Esta función es simple y útil para enseñanza porque muestra con claridad la idea de derivada discreta. Sin embargo, exige que T_s sea constante y distinto de cero.

Variable	Significado
now	Cuenta actual del encoder leída desde TIM2->CNT.
last	Cuenta de la llamada anterior; se almacena como variable estática.
delta	Incremento de ticks entre dos muestras consecutivas.
T_s	Periodo de muestreo en segundos.
delta / T_s	Velocidad expresada en ticks por segundo.

4.2 Conversión a RPM dentro de `TIM4_IRQHandler`

En la práctica principal, la velocidad del motor no se calcula con `encoder_get_speed_tps()`. En su lugar, la ISR de TIM4 lee la posición acumulada cada 10 ms y aplica una fórmula de conversión específica del sistema:

$\text{rpm} = \text{pulsos} / 44 * 100 * 60 / 4.4$

- **44** es la cantidad de pulsos por vuelta directa en el eje del motor.
- **100** convierte de una ventana de 10 ms a 1 segundo.
- **60** convierte de revoluciones por segundo a revoluciones por minuto.
- **4.4** representa la relación de engranajes indicada en el comentario del proyecto.

Después de calcular la velocidad, el programa limpia el acumulado con `quad_STM32.reset()`. Así, la siguiente ventana de 10 ms vuelve a empezar desde cero.

5. Integración con la aplicación principal

El flujo real del proyecto muestra por qué este módulo es esencial dentro del sistema completo:

Etapas	Uso del módulo de encoder
Arranque	Se llama a <code>quad_STM32.init()</code> después de inicializar el reloj y antes de arrancar la adquisición de datos.
Muestreo periódico	TIM4 interrumpe cada 10 ms y lee <code>quad_STM32.get_count()</code> .
Conversión física	Los pulsos de la ventana se transforman a RPM del motor.
Preparación para RLS	La RPM alimenta el historial y luego se usa en el vector de salida del modelo ARX.
Reinicio de ventana	Se ejecuta <code>quad_STM32.reset()</code> para comenzar la siguiente medición.

```
quad_STM32.init();
```

```
...
```

```
g_motor.pos = quad_STM32.get_count();  
g_motor.rpm = g_motor.pos / 44.0f * 100.0f * 60.0f / 4.4f;  
g_motor.pos = 0;  
quad_STM32.reset();
```

Didácticamente, este punto es muy valioso: el módulo del encoder no está aislado. Forma parte de una cadena completa de medición que termina influyendo en la identificación paramétrica del motor.

6. Observaciones y notas para clase

- **Ventaja principal:** usar TIM2 en hardware evita perder pulsos y elimina la necesidad de procesar interrupciones por cada flanco del encoder.
- **Rango amplio:** TIM2 es de 32 bits en STM32F405, por eso puede contar mucho antes de desbordarse.
- **Filtro digital:** el valor 4 en IC1F/IC2F es una decisión práctica para reducir ruido; no siempre existe un valor universal.
- **Posible mejora:** validar que T_s sea mayor que cero dentro de `encoder_get_speed_tps()` para evitar divisiones inválidas.
- **Posible mejora didáctica:** documentar explícitamente si el encoder entrega 44 pulsos por canal, por ciclo de cuadratura o por vuelta efectiva, porque eso cambia la conversión física.
- **Coherencia del sistema:** el comentario en `main.c` y la configuración de TIM4 apuntan a una ventana de 10 ms, no de 1 ms. Conviene remarcarlo al explicar el proyecto.

Conclusión: `quad_enc` es el puente entre el fenómeno físico de rotación y la representación digital que usa el firmware. Sin este módulo no sería posible medir posición, estimar velocidad ni alimentar de forma confiable al algoritmo RLS/ARX.